Unit-2

Convolutional Neural Network

(1) Why Not use ANN for image classification?

→ ANN can classify images, but it become inefficient for large images. There are too many parameters & loss of spatial features.

Ex:

small RGB img : 100 × 100 × 3 = 30,000 inputs

if ANN 1st hidden layer has 1000 neurons, so

weight = 30000 × 1000 = 30,000,000

→ This makes training slower, memory-heavy and more prone to overfitting.

problem:

→ ANN usually convert an image into a flat vector:

- After flattening the network doesn't explicitely preserve the 2D relationship b/w nearby pixels. but in images, nearby pixels are important for detecting edges, eyes, ear, shapes etc.

CNN

→ CNN don't try to understand the whole image at once they look at small patches and slowly build up the full picture.

(1) Pixel: (Picture Ele) is the smallest unit of an img. img containing thousands or millions of pixels.

Ex: a grey scale img : 28 × 28 = 784 pixels
(height, weight)

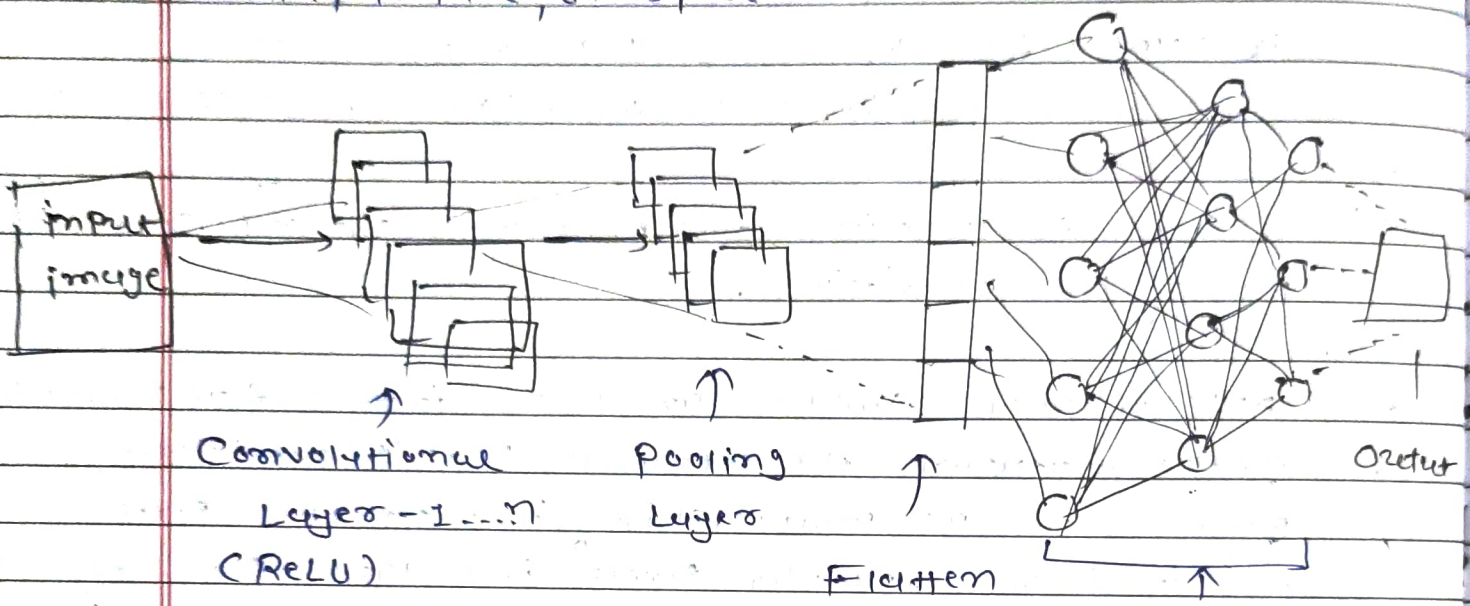
(2) Channel: One layer of image into

Grey scale : (28, 28, 1) → only 1 channel

RGB : (100, 100, 3) → 3 channel called RGB

CNN Architecture

→ The arrangement of different layers that process an img step-by-step. Early layer simply learn edges, while deeper layer learn complex pattern such as shape, texture, faces, eye etc.



(1) Convolutional Layer

→ core operation of CNN, how CNN looks at small regions of an image and features.

1) kernel: (Filter) is a small matrix that slides over an image and looks for a particular visual pattern

Ex: 1 0 0 -1
1 0 -1
1 0 -1

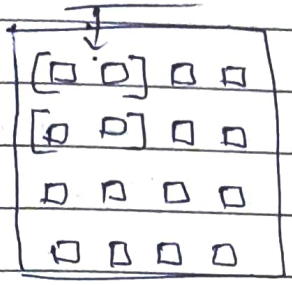
Feature Map = $(n - m + 1)$

classmate
Date
Page

-> Usually do not manually create these ~~features~~ filters. Their values are learned automatically during training through backpropagation.

2> Convolutional: It is the operation where the filter slides across the image, perform element wise multiplication with pixels add the result and produce value.

Filter



-> multiply + Add

↓

O/P

=> Then filter move to the next position and repeats the calculation.

3> Feature Map: After filter has scanned entire image, all the calculated value from a new matrix called Feature Map.

-> It tells, where the feature detected by that filter appears in the image.

code: Conv2D (filters = 32, kernel_size (3,3), activation = 'ReLU')

0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	"	"	"
0	0	0	"	"	"

Image

X

-1	0	1
-1	0	1
-1	0	1

Filter

=>

0	255	255	0
0			0
0			0
0			0

Feature Map

6 x 6 X 3 x 3 = 4 x 4

28 x 28 X 3 x 3 = 26 x 26

(4) Stride: It means how many pixels the filter moves at each step while scanning the image.

=> Stride-1: filter moves slowly, it captured more detailed information and produce large feature map.

=> Strid-2: It means fewer convolution operations, so a smaller o/p feature map.

(5) ~~Padding~~ Padding: Adding extra pixels around the border of the input, usually zero(0).

-> Normal convolution, the filter cannot move beyond the image boundaries, b'coz of this the o/p became smaller than the original image.

Ex:

input = 5x5
Filter = 3x3 => o/p = 3x3
Stride = 1

-> if we repeatedly apply convolution, the feature maps keep shrinking.

=> code: padding = 'valid' # No padding
padding = 'same' # padding (same height x weight)

Ex: 5x5 X 3x3 = 3x3

↓

7x3

$n - m + 1 = n$

$n - 3 + 1 = 5$

$n = 7$

$\therefore n - m + 1$

$= 7 - 3 + 1$

$= 5 \times 5$ ← o/p same as h x w

(2) Pooling

→ Used to reduce the size of feature maps while keeping the important information.

Why used?

→ Reduce dimension - Smaller feature maps, faster computation

→ Extract dominant - Keep most imp info, ignore noise features

→ Make model translation - Position of object doesn't invariant matter much

(1) Max Pooling : Takes the largest value from each small region of feature map

ex: $\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$ $\begin{bmatrix} 8 & 9 \\ 7 & 6 \end{bmatrix} \begin{bmatrix} 5 & 8 \\ 4 & 1 \end{bmatrix}$ $\Rightarrow \begin{bmatrix} 6 & 4 \\ 9 & 8 \end{bmatrix}$

→ Use when detecting strong/dominent feature like edges, textures, parts.

→ Most used in CNN.

(2) Average Pooling: It calculate avg of all values

ex: $\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix} \Rightarrow \frac{1+3+5+6}{4} = 3.75$

→ Use when you want smoother, less noisy representation eg. in early layer or for reducing sensitivity

→ Less used, but good for preserving background info.

(3) Min pooling: Take min value

→ Use when highlightening dark features, anomalies, or background suppression. (rare)

(3) Flatten Layer → It only changes the shape of data

→ The output still a 3D feature map, A Dense layer expects a 1D vector, so flatten converts the 3D feature maps into one long 1D vector.

Ex:

$4 \times 4 \times 32 = 512$. [0.2, 0.8, 0.1, 0.7 ...]

(4) Fully connected Layer :

→ basically the ANN part inside the CNN. every neuron connected to all O/P from previous layer.

* Full workflow

→ Step-1 : input image (28 x 28 x 1)
height weight channel

→ Step-2 : First convolution - 3 x 3, 32 filters

=> apply 32 different filters, each having spatial size 3x3

28 x 28 x 1

↓ (32 filters)

28 x 28 x 32

→ learn edges, lines, curve

→ so 32 feature maps created

Step-3 : Max Pooling - (2×2)

28 × 28 × 32 → (2×2) → 14 × 14 × 32

reduce height, weight, but channel remain same,

Step-4 : second convolution - 3×3, (64 filters)

14 × 14 × 32 → (64 filters) → 14 × 14 × 64

More complex pattern from previous convo

Step-5 : second Max pooling

14 × 14 × 64 → (2×2) → 7 × 7 × 64

Step-6 : Flatten (3D → 1D)

7 × 7 × 64 = 3136

Step-7 : Dense layer → passed to the ANN

3136 features → Dense (128, Relu) → 128 neurons

Step-8 : O/P layer (10 neuron + Softmax)

=> MNIST has 10 classes : 0 1 2 3 4 5 6 7 8 9

0 - 0.01

1 - 0.05

2 - 0.02

3 - 0.03

:

7 - 0.89

prediction = 7